

M1.1: JVM GC Tuning

Theory: Tunes JVM heap generation ratios, garbage collector choice (G1/ZGC), and safe-point flags to reduce GC pauses.

Details: Adjusts '-XX:NewRatio' and G1 maximum pause target '-XX:MaxGCPauseMillis'. Minimizes old generation promotions and leverages ZGC concurrent phase markers.

Trade-off: Shorter pause targets require higher CPU thread availability for concurrent GC workers, potentially degrading runtime throughput.

M1.2: Go Allocator & pprof

Theory: Tracks Go memory escape traces and runtime heap profiles to reduce heap allocation pressures.

Details: Inspects escape logs via '-gcflags="-m"'. Binds heap variables to stacks where possible to bypass GC write-barrier sweeping.

Trade-off: Escape analysis enforces strict stack sizes, occasionally limiting the use of pointer-based abstractions.

M1.3: Python Async Loop Profiling

Theory: Identifies blocking synchronous I/O operations inside asyncio event loops, offloading blocking tasks to thread executors.

Details: Leverages 'loop.set_debug(True)' to capture loop delay warnings. Delegates blocking DB transactions to 'run_in_executor()'.

Trade-off: Spawning executor thread pools introduces memory footprints and context switches.

M1.4: heaptrack Leaks

Theory: Monitors C/C++ or native heap operations (malloc/free) to isolate memory leaks and allocation patterns.

Details: Traces physical memory drift profiles, showing leaky calls and backtraces.

Trade-off: Profiling native allocators incurs memory tracing footprints, limiting long-term use in production.

M1.5: JVM CPU 100%

Theory: Identifies JVM CPU bottlenecks by mapping high-cpu OS threads to Java thread dump names.

Details: Finds high-CPU OS threads via 'top -Hp <pid>'. Converts thread LWP IDs to hex format, matching them with 'nid' headers in 'jstack'.

Trade-off: Capturing thread dumps momentarily freezes the JVM; profiling must be limited to short bursts.

M1.6: Go GC write barrier

Theory: Evaluates Go concurrent tri-color marks and compiler write barriers during GC runs.

Details: Avoids memory allocations during GC write-barrier cycles, reducing runtime overheads.

Trade-off: Write barriers protect memory safety at the cost of short mutator write delays.

M2.1: Thread Pool Formula

Theory: Sizes thread pool dimensions based on CPU-bound $(N + 1)$ or I/O-bound $(N \times U \times (1 + W/C))$ task natures.

Details: Computes thread pools based on Wait-to-Compute ratio assessments to avoid scheduling blockages.

Trade-off: High pool counts saturate context scheduler limits; low pool counts choke processing rates.

M2.2: Flame Graph

Theory: Visualizes CPU stack samples dynamically, using block widths to represent hot path durations.

Details: Gathers CPU frame samples via 'perf' or 'async-profiler', rendering horizontal call stacks in interactive SVGs.

Trade-off: High sampling rates overhead CPU; typical setups limit profiling to 1-2 minutes.

M2.3: Cacheline align

Theory: Groups variables inside 64-byte boundaries to avoid CPU false sharing overheads.

Details: Prevents cacheline invalidations by wrapping concurrent variables with '@Contended' annotations or padding fields.

Trade-off: Pad bytes consume unused stack and heap memory, increasing memory usage.

M2.4: CPU Affinity

Theory: Restricts thread migration behaviors by pinning execution threads to dedicated CPU cores.

Details: Configures thread affinities using 'sched_setaffinity' system calls, preventing L1/L2 cache misses.

Trade-off: Locks task threads to specific hardware, reducing system flexibility during unexpected surges.

M2.5: Context Switch

Theory: Identifies voluntary (waiting for locks) and involuntary (CPU time-slice expired) context switches to reduce thread scheduling costs.

Details: Tracks switch frequencies using 'vmstat'. Employs CAS atomic loops or lock-free queues to reduce scheduling interrupts.

Trade-off: CAS polling consumes CPU cycles, requiring spin limits before sleeping threads.

M2.6: JVM Safepoint

Theory: Identifies Safepoint pauses triggered by garbage collections or bias lock revocations.

Details: Monitors safepoint delays via '-XX:+PrintGCApplicationStoppedTime' log reviews.

Avoids long counting loops that miss safepoint checks.

Trade-off: Inserting frequent safepoint checks slows down tight loop operations.

M3.1: B+Tree Selection

Theory: Reduces disk block reads by optimizing index structures and avoiding secondary index 回表 lookups.

Details: Measures index selectivities. Builds composite indexes or covering indexes to fulfill queries without clustered B+Tree traversals.

Trade-off: Excess indexes increase database write overheads and consume disk space.

M3.2: MySQL Lock Deadlock

Theory: Analyzes Shared (S) and Exclusive (X) lock deadlocks on gap and next-key ranges.

Details: Inspects deadlock records via 'SHOW ENGINE INNODB STATUS'. Aligns lock request orders in code to avoid circular dependencies.

Trade-off: Strict locking order increases query wait queues, potentially reducing throughput.

M3.3: HikariCP pool

Theory: Optimizes database pool connections based on physical disk spindle counts and core processing capacities.

Details: Computes pool scale boundaries using the formula: $connections = ((core_count \times 2) + effective_spindle_count)$.

Trade-off: High connection limits overload DB thread schedulers; low limits throttle app queries.

M3.4: Redis Slow Big Key

Theory: Identifies O(N) operations and bulky keys to prevent Redis single-thread blocking.

Details: Captures high-latency queries using 'SLOWLOG GET'. Splits large hashes and triggers 'UNLINK' for async deletions.

Trade-off: Splitting keys complicates queries, requiring multi-key lookups.

M3.5: Redis LFU Sample

Theory: Implements frequency-based eviction strategies using logarithmic counters.

Details: Compares standard LRU evictions with frequency-based LFU (Least Frequently Used) selections to handle short-lived hot key surges.

Trade-off: LFU adds decay time calculation costs to memory evictions.

M3.6: CQRS separating

Theory: Decouples write models from read views, using event buses to sync read-only databases.

Details: Isolates PostgreSQL write tables, forwarding updates via CDC channels to Elasticsearch read indices.

Trade-off: Introduces final consistency delay windows, requiring UI optimizations.

M4.1: Page Cache Flush

Theory: Controls kernel dirty page syncs to prevent system I/O freezes.

Details: Fine-tunes 'vm.dirty_background_ratio' (starts background flushing) and 'vm.dirty_ratio' (forces synchronous writing) in '/etc/sysctl.conf'.

Trade-off: Frequent background flushing increases write activity but ensures low latencies.

M4.2: Disk IO Scheduler

Theory: Matches storage hardware with appropriate I/O scheduling algorithms (BFQ, Kyber, mq-deadline, or None).

Details: Sets the None scheduler for NVMe drives to bypass software queuing; uses mq-deadline for HDDs.

Trade-off: Misconfigured schedulers degrade I/O rates, increasing disk read latencies.

M4.3: TCP Socket Window

Theory: Scales TCP window parameters based on Bandwidth-Delay Products (BDP) to maximize throughput.

Details: Adjusts socket memory settings: 'net.ipv4.tcp_rmem' and 'net.ipv4.tcp_wmem' to utilize high-bandwidth networks.

Trade-off: Bulky socket buffers consume host RAM, limiting max concurrent connection 水位的 scaling.

M4.4: Zero-copy sendfile

Theory: Routes file transfers directly from read buffers to network cards, bypassing user-space allocations.

Details: Employs 'sendfile' or 'mmap' calls, reducing data moves and system call contexts from 4 to 2.

Trade-off: Limits intermediate payload mutations (e.g. gzip compression) during transfer.

M4.5: TIME_WAIT reuse

Theory: Recycles ephemeral ports safely by managing sockets in the 2MSL TIME_WAIT state.

Details: Enables 'net.ipv4.tcp_tw_reuse' to reuse TIME_WAIT sockets for outgoing connections.

Trade-off: Requires timestamp checks ('net.ipv4.tcp_timestamps=1') to prevent packet overlaps.

M5.1: Async IO vs Thread

Theory: Compares non-blocking epoll event loops with traditional thread-per-connection architectures.

Details: Evaluates event loop memory footprints and connection watermarks.

Trade-off: Event loops handle millions of idle sockets but block on CPU-heavy tasks.

M5.2: Epoll Thundering Herd

Theory: Avoids wake-up stampedes when incoming connections trigger multiple listening threads.

Details: Deploys 'EPOLLEXCLUSIVE' flags or enables 'SO_REUSEPORT' socket reuse.

Trade-off: Sockets require separate thread assignment configurations.

M5.3: SIMD Vector

Theory: Runs parallel loop computations using AVX hardware registers.

Details: Instructs compilers to auto-vectorize loops, loading 256-bit registers with data blocks.

Trade-off: Demands hardware instruction compatibility.

M5.4: APM latency trace

Theory: Profiles microservice latencies by tracing database queries, serialization times, and network delays.

Details: Traces span durations across services, pinpointing database query bottlenecks.

Trade-off: High tracing resolutions overhead CPU and network resources.

M5.5: Lock Reduction AQS

Theory: Compares AQS enqueue/dequeue overheads with LMAX Disruptor CAS lock-free ring buffers.

Details: Replaces standard mutex locks with lock-free CAS ring buffers to resolve thread contention.

Trade-off: CAS loops burn CPU cycles under high contention, requiring spin limits.

Zone T1: Daily Diagnostics Core Libraries

async-profiler, perf tool, HikariCP pool, Intel AVX-512, LMAX

Disruptor, POSIX sendfile, sysctl.conf

Zone T2: System Faults & Diagnostics

cascading_thread_pool_exhaustion_panic,
dirty_data_write_skew_migration_error, idem-
potent_duplicate_submission_intercept, cir-
cuit_breaker_open_fast_fail_triggered, poi-
son_pill_max_retry_exhaust_dlq_redirect, watch-
dog_lease_renewal_lock_expiration